The University of Texas at Arlington

EE5314 Embedded Microcontroller Systems

Final Project

Submitted By:

Tirth Shah (#1002210113)

Contents

1. Introduction 3
2. Objective 3
3. Parts, Pins and Components 3
   3.1 Physical Components 3
      3.1.1 Microcontroller 3
      3.1.2 DC Motor and Encoder 4
      3.1.3 LCD Display 4
   3.2 Pins and Configuration 4
4. Project Approach 5
5. Results & Discussion 7

1. Introduction

Embedded systems are an integral part of modern technology, playing a crucial role in bridging the gap between software and hardware. It has diverse applications ranging from home automation and healthcare devices to automotive systems and industrial automation. These systems are designed to perform specific tasks with efficiency, reliability, and precision. A prime example of their real-world importance is in control systems, where embedded microcontrollers govern the behaviour of motors, sensors, and actuators to ensure optimal performance in various applications.

2. Objective

The objective of this project is to successfully integrate a DC motor with encoder and a LCD display with a microcontroller and implement a control system to drive the motor and set the RPM of the motor to custom values that is hard coded as well as user controlled through a UART bridge. Timers are used for precise timing and synchronization and controlling the simulation rate.

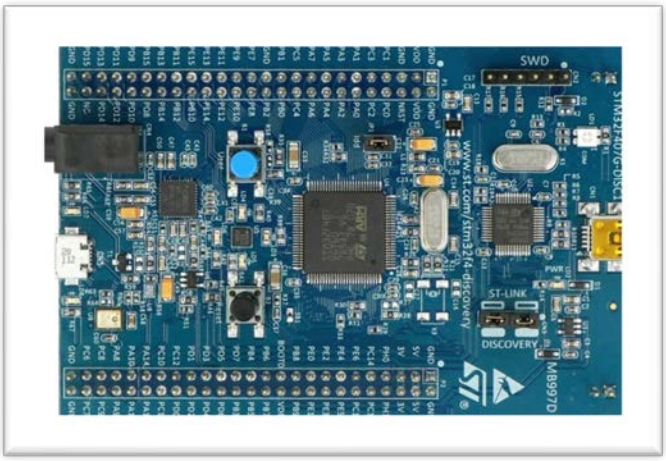
3. Parts, Pins and Components

3.1 Physical Components

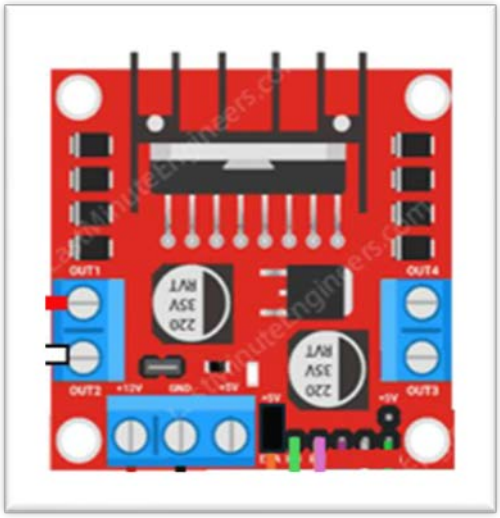
3.1.1 Microcontroller

The **STM32F407G-DISC1** discovery board from STMicroelectronics was used for this project.

Bus Clock used: 8 MHz (APB1)



3.1.2 DC Motor and Encoder



3.1.3 LCD Display

The LCD used in this project was HD44780. It has two rows capable of displaying 16 characters per row. It has a backlit display with 16-pin parallel surface which already comes with I2C adapter.

3.2 Pins and Configuration

a. TIM1: this was enabled in the encoder mode to get the encoder counts
b. TIM7: belonging to the APB1 bus with boundary address 0x4000 1400 - 0x4000 17FF. The prescaler and counter period was set to 99 and 799 respectively. This timer was used to tune the PI controller according to a simulation rate of 0.01s.
c. USART2: this is the connectivity pin that is used to receive and transmit the data between the computer and microcontroller to send commands using a USB to USART TTL adapter. This pin in enabled in asynchronous with interrupt mode.
d. I2C: it is the communication interface between the microcontroller and the LCD.

4. Project Approach

The approach to this project was in a constructive and simplistic manner. The motor was allowed to run as per the instructions from the user. The motor was controlled using a control system in a closed loop implementation.

The command i.e. the desired RPM is passed on using UART2 pin. Anything that was displayed on the SmartTTY terminal or the serial plotter was done using this port. In the closed loop approach, the command was passed to the PI controller instead of the motor directly. The pulse was set using the timer and the simulation rate was decided. It was passed through the PWM driver for precision control. The PI controller then adjusts the motor speed. The measured RPM and commanded RPM are displayed and the error was calculated.

A proportional-integral (PI) controller was used specifically for this experiment for certain reasons. This controller helps to eliminate the steady state errors caused by the model. It also ensures fast response to changes and maintains long-term accuracy without oscillations. Also, it is much simpler to implement, thus reducing complexity. The The response generated by the PI control is represented by:

$$D(s) = K_p + K_i \frac{1}{s}$$

Converting it into the z-domain, the resultant difference equation looks like:

$$u[k] = u[k-1] + K_p(e[k] - e[k-1] + K_i e[k]$$

Where e[k] is the error, u[k] is the control output and other are the control parameters. This difference equation is implemented using a periodic interrupt using a timer.

```
p = p_old + Kp*( error - error_old) + Ts*Ki*error_old;  //PI Controller
p_old = p;
error_old = error;
```

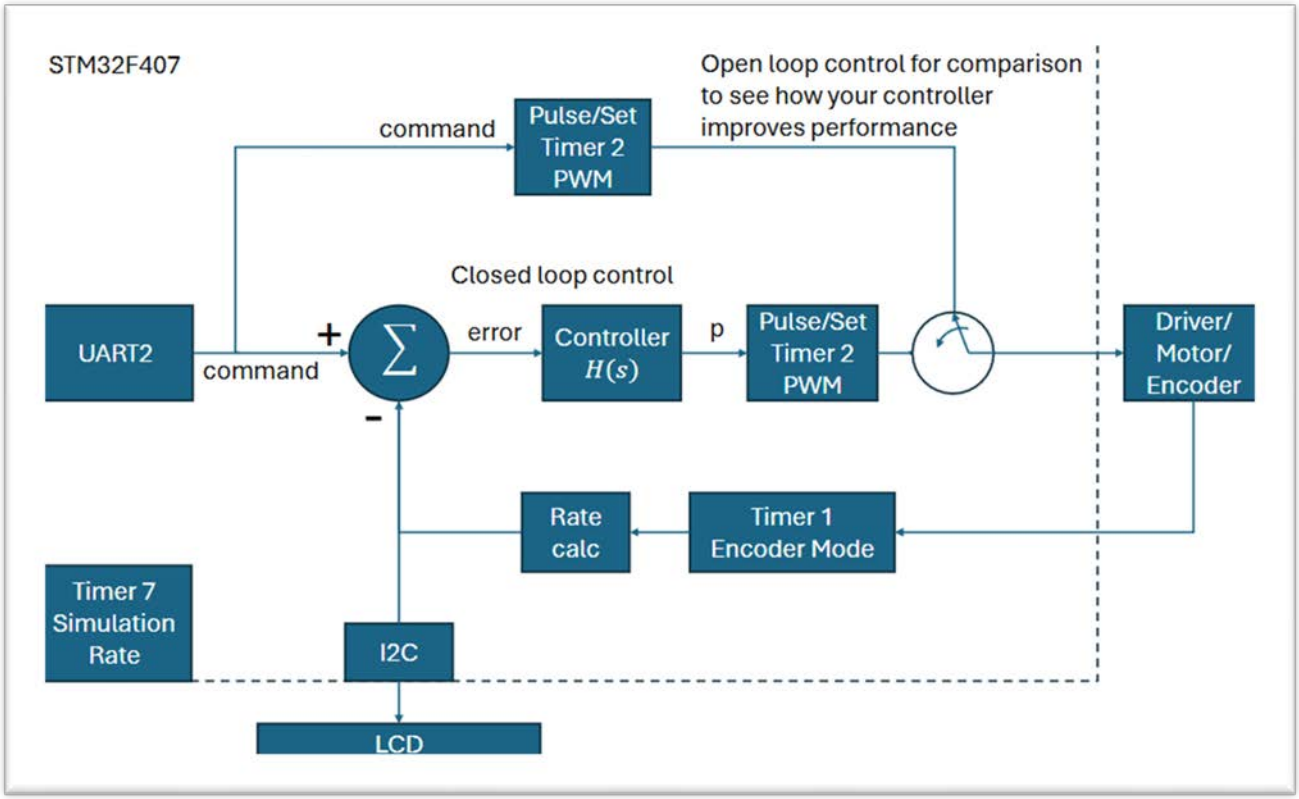
Implementing the PI controller in the code along with the error.

The RPM was calculated from the angular velocity of the motor rotating and it was converted into RPM as follows:

```
angle = (float)(count) * (360.0 / (float)enc);

temp2 = angle;
rate = ((temp2 - temp1) / Ts) * (60.0 / 360.0); //in rpm
temp1 = temp2;
```

The LCD screen was included as well. The function to put the cursor to the desired column and row for display, sending the string and the initialization function was written.



This block diagram represents the flow of the entire project.

The console menu was printed using the UART interface. The menu looked something like this:

```
void menu(void) {

    char menu[] =   "1. Turn on Motor at 5 RPM\r\n"
                    "2. Set to 30 RPM\r\n"
                    "3. Set to 80 RPM\r\n"
                    "4. Set to 130 RPM\r\n"
                    "5. Custom RPM (0-150)\r\n"
                    "6. Turn off the Motor\r\n";

    HAL_UART_Transmit(&huart2, (uint8_t*)menu, strlen(menu), HAL_MAX_DELAY);


}
```

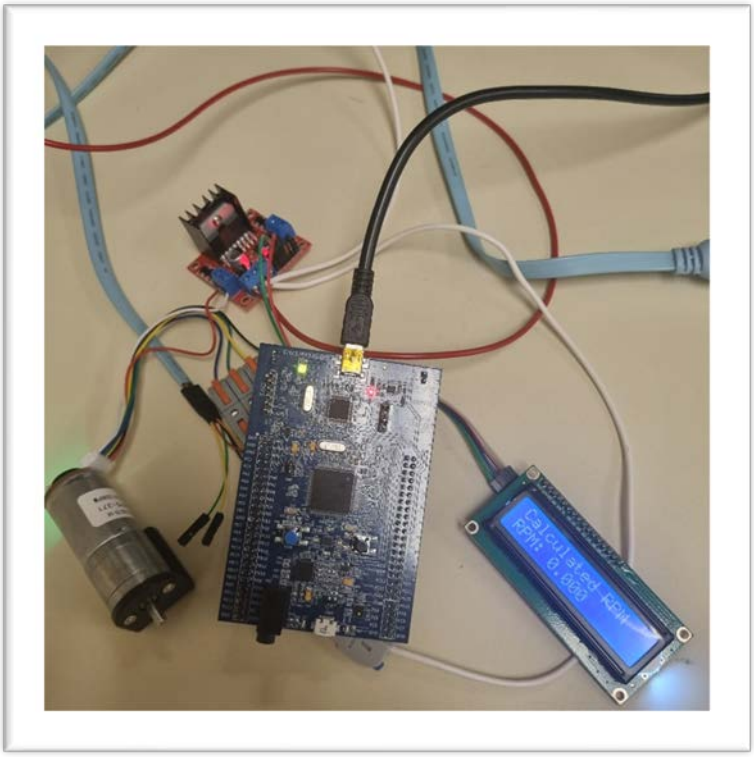
The UART callback function that was implemented had different commands as per the inputs received from the user. Each option entered by the user triggered a certain function. For example, when a user enters 3, a command was passed to the controller that the RPM value is set to 80. The error was then calculated and the controller passes the calculated RPM to the motor through the timer and PWM generation channel and the output was shown to be in the range of 78-82 giving an error of approximately ±0.01.

Similarly, for the custom RPM functionality, the message to enter the RPM was transmitted using UART. The input from the user was recorded and sent to the controller for the implementation as discussed earlier.

The LCD display displayed the actual RPM that is the RPM calculated after the scope of error. Hence, it is observed that when the command RPM is printed, the value is constant which was given by the user. Whereas in the calculated RPM, it varies in a range because of the error, typically in the range of ±2 RPM.

5. Results & Discussion

As discussed earlier, the objective of implementing a control system to control the motor at the RPM desired by the user was successfully achieved. This is a picture from the actual implementation and the hardware setup of the experiment.



One thing that was noticed during the implementation was that for the case of custom RPM, when the user inputs a RPM value, the system took 5-10 duty cycles to implement that RPM. Hence, the motor would stop and suddenly start with the desired output.

In summary, the use and operation of the STM32F407 was learned, including writing the drivers and was demonstrated by implementing this project which had various aspects like USART, timers, interrupts, control systems, I2C, encoders and LCD.

----------End of Report----------